

Lesson 13: Managing State at Scale

The problem with setState

In Lesson 4 you learned `setState()`: keep data in a `StatefulWidget`, call `setState()` when it changes, and Flutter rebuilds the screen. For one screen, this works well.

But real apps have many screens and many widgets. Soon you face a problem: two widgets need the same data, but they live in different parts of the tree. A cart badge in the app bar and a product list on the home screen both need to know how many items are in the cart.

With only `setState()`, you have one option: keep the data in a shared parent and pass it down. This is called lifting state up.

Lifting state up

You move the data to the lowest widget that is above *all* the widgets that need it:

```
class ShopApp extends StatefulWidget {
  const ShopApp({super.key});

  @override
  State<ShopApp> createState() => _ShopAppState();
}

class _ShopAppState extends State<ShopApp> {
  int cartCount = 0;

  void addToCart() {
    setState(() {
      cartCount++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(title: Text('Cart: $cartCount')),
        body: ProductList(onAdd: addToCart),
      ),
    );
  }
}
```

The parent holds `cartCount`. It passes the value down to the app bar, and passes the `addToCart`

callback down to the product list. When the button is tapped, the callback runs, the parent updates, and everything rebuilds.

This is a clean pattern. For small apps, it is all you need.

Passing callbacks down

The child widget does not own the data. It only gets a function to call:

```
class ProductList extends StatelessWidget {
  final VoidCallback onAdd;

  const ProductList({super.key, required this.onAdd});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: onAdd,
      child: const Text('Add to cart'),
    );
  }
}
```

VoidCallback is just a short way of writing "a function that takes no input and returns nothing". The child calls it, the parent does the real work.

When this pattern breaks

Lifting state up works -- until your app grows. Then three pains appear:

1. Prop drilling. The data must pass through every widget in between, even ones that do not care about it. Add a grandchild, a great-grandchild, and your constructors fill up with values they only pass along.
2. Rebuilds. One `setState()` rebuilds the whole parent and everything below it. With a big tree, this gets slow.
3. Shared across screens. If the cart page and the home page both need the cart, where is the "shared parent"? It has to sit above *both* screens -- often at the very top of the app. Now one widget holds data for the whole app, and every change rebuilds everything.

This is the point where you need a real state management tool. Lesson 14 shows you the simplest one: Provider.

Key terms

- Lifting state up -- moving shared data to a parent widget that sits above all widgets that need it.
- Callback -- a function passed down to a child so the child can tell the parent something happened.
- Prop drilling -- passing data through many layers of widgets that do not use it, only to reach a deep child.
- State management -- tools and patterns for sharing data across a whole app without messy passing.